Utah State University
ECE 6010
Stochastic Processes, Fall 2025
Programming Exercise # 2

Introduction

This exercise will provide an opportunity to do some calculations and plots with actual data. The intent is to make some of the abstract concepts a little more concrete. This exercise is to be done using MATLAB or Python.

Background

`randn` Every call to the MATLAB function `randn` generates an independent instance of a standard Gaussian random variable. That is, `randn` is a Gaussian random number generator: $X \sim \mathcal{N}(0,1)$. Random column vectors of length `n` are generated by `randn(n,1)`. Row vectors are generated with `randn(1,n)`. A matrix of random numbers is generated with `randn(n,m)`. For more information, type `help randn` in MATLAB.

Python: Using `random` module: `random.normalvariate()` give a $\mathcal{N}(0,1)$ random number.

Python: Using `numpy` module: The function `numpy.random.randn(n,m)` generates an $n \times m$ numpy array of $\mathcal{U}(0,1)$ random numbers.

You can also use `numpy.standard_normal` or `numpy.normal`.

`rand` Similarly, the MATLAB function `rand` generates independent uniform $\mathcal{U}(0,1)$ random numbers. Column and row vectors and matrices of random numbers are generated using `rand(n,1)`, `rand(1,n)`, `rand(n,m)`. For more information type `help rand` in MATLAB.

In Python: Using the `random` module: The function `random.random()` generates uniform $\mathcal{U}(0,1)$ random numbers.

Using `numpy` module: The function `numpy.random.rand(n,m)` generates an $n \times m$ numpy array of $\mathcal{U}(0,1)$ random numbers.

Histograms The MATLAB `histogram` command produces a histogram. This is a representation of the empirical density function. In a histogram, a sequence of bins is established. For each value in a set of data, the number of times that data points fall in a bin is counted. In the MATLAB `histogram` command, the histogram is plotted automatically. To see an example of how the histogram works, type the following in MATLAB:

```
x = randn(1,1000);    % create a row vector of
                      % 1000 Gaussian random numbers
histogram(x,20);      % plot the histogram with 20 bins
```

```
histogram(x,100);    % plot the histogram with 100 bins
```

For more information, type `help histogram` in MATLAB.

Python: the `numpy.histogram` function computes histograms.

```
import numpy as np
import matplotlib.pyplot as plt
x = np.random.randn(1,10000)
hist, bins = np.histogram(x, bins = 100)
plt.plot(bins[:-1],hist)

# Alternatively, if you just want to plot the histogram:
plt.hist(x.T, bins=100)
```

Estimating mean and covariance Given a sequence of vector observations $\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_N$, where each vector is a column vector of length n drawn independently and identically distributed according to some distribution, the sample mean of the distribution is

$$\hat{\boldsymbol{\mu}} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{x}_i.$$

The sample covariance is

$$\hat{\Sigma} = \frac{1}{N-1}\sum_{i=1}^{N} (\mathbf{x}_i - \hat{\boldsymbol{\mu}})(\mathbf{x}_i - \hat{\boldsymbol{\mu}})^T.$$

In MATLAB, if `X` is a matrix whose columns contain values of a random vector, such as the 2-dimensional random vectors

$$X = \begin{bmatrix} 1.8304 & 0.5409 & -0.3631 & -0.8211 \\ 2.6978 & 1.6102 & 0.3355 & 2.2543 \end{bmatrix}$$

then `mean(X,2)` will compute the sample mean and `cov(X')` will compute the sample covariance. Notice the transpose on the `X`!

In Python, if `X` is a matrix whose columns contain values of a random vector, such as

```
X = np.random.randn(2,1000)
```

then `np.mean(X,1)` will compute the sample mean and `np.cov(X)` will compute the sample covariance.

Exercises

Basic Random Variables and Transformations

1. (5 pts) Generate 1000 points of a $\mathcal{N}(2, 5)$ random variable. (That is, you will need to take an $\mathcal{N}(0, 1)$ random variable and modify it to be a $\mathcal{N}(2, 5)$ random variable.) Plot the histogram of the data. Plot the histogram *so that it is scaled like a pdf* (so it integrates to 1). Overlay the plot of the pdf on the scaled histogram. Estimate the mean and the variance from the data you generate. How closely do the estimates correspond to actual parameters? Comment on how much the histogram looks like the pdf.
2. (5 pts) Generate 1000 points of a $\mathcal{U}(-2, 5)$ random variable. (That is, you will need to take a $\mathcal{U}(0, 1)$ random variable and modify it to be a $\mathcal{U}(-2, 5)$ random variable.) Plot the histogram of the data. Estimate the mean and the variance from the data you generate. How closely do the estimate correspond to the actual parameters? With a scaled histogram, overlay the pdf of the random variable. Comment on the similarity of the histogram and the pdf.
3. (20 pts) (5 pts) Write a MATLAB or Python function that will generate N points of Gaussian $\mathcal{N}(\boldsymbol{\mu}, \Sigma)$ data, where $\boldsymbol{\mu}$ is a vector of length n and Σ is the $n \times n$ covariance matrix. The function should have the "declaration"

   ```
   function X = gengauss(mu,Sigma,N)
   ```

   or

   ```
   def gengauss(mu,Sigma,N)
   ```

   in Python.

   Test your function by generating 1000 points of a $\mathcal{N}(\boldsymbol{\mu}, \Sigma)$ random variable, where

   $$\boldsymbol{\mu} = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \qquad \Sigma = \begin{bmatrix} 1 & 1 \\ 1 & 3 \end{bmatrix}$$

   With this data, do the following:

   - (2 pts) Estimate the mean vector and the covariance matrix using these 1000 generated points. Verify that the they are "reasonable" estimates of the desired mean and variance.
   - (2 pts) Plot a *scatter plot* of the data, by plotting the first coordinate against the second coordinate as points:

     ```
     mu = [1;2];                      % mean vector
     Sigma = [1 1; 1 3];              % covariance matrix
     X = gengauss(mu,Sigma,1000);     % generate the data using your func
     plot(X(1,:),X(2,:),'.','MarkerSize',1);  % scatter plot
     axis('equal');                   % adjust axis portrayal
     hold on;                         % save for future plots
     ```

     The arguments `'MarkerSize',1` make the plotted dots be small.

     Python:

```
mu = np.array([1,2])
Sigma = np.array([[1, 1],[1, 3]])
X = gengauss(mu,Sigma,1000)
plt.scatterX[0,:],X[1,:],s=0.5, marker='.')
plt.axis('equal')
```

- (3 pts) Also plot the directions of the major and minor axes.

```
[u,v] = eig(inv(Sigma));        % eigendecomposition
v1 = u(:,1)/sqrt(v(1,1));       % first direction
v2 = u(:,2)/sqrt(v(2,2));       % second direction
plot([mu(1),mu(1)+v1(1)],[mu(2),mu(2)+v1(2)]);
plot([mu(1),mu(1)+v2(1)],[mu(2),mu(2)+v2(2)]);
```

- (3 pts) Plot contours of the pdf. The function `plotellipse.m` (on the class website) may be helpful, overlayed on the scatter plot.
- (5 pts) Compare the axes of the ellipses with the eigenvectors of Σ. What is the relationship?

4. (35 pts) The file `prog1dat.mat` (on the class website) contains $N = 1000$ data points representing measurements x_1, x_2, x_3, x_4 from a four-dimensional physical system. Load the data into MATLAB using the command

   ```
   load prog1dat
   ```

   A 4×1000 variable `X` is loaded with the data in it; each column represents one observation of the four-dimensional data.

   Python: You need the `loadmat` module. If you haven't already done so, install `scipy`:

   ```
   pip install scipy
   ```

   Then:

   ```
   from scipy.io import loadmat
   data = loadmat('prog1dat.mat')
   X = data['X']   # pull out the numerical data
   ```

   - (5 pts) Let $\mathbf{X}^{(1)}$ be obtained from the first two components of the four-dimensional data,

     $$\mathbf{X}^{(1)} = \begin{bmatrix} X_1 \\ X_2 \end{bmatrix}.$$

     Estimate $\boldsymbol{\mu}^{(1)}$ (the mean vector) and Σ_{11} (the covariance matrix of $\mathbf{X}^{(1)}$).
     Let

     $$\mathbf{X}^{(2)} = \begin{bmatrix} X_3 \\ X_4 \end{bmatrix}$$

     Estimate $\boldsymbol{\mu}^{(2)}$ and Σ_{22} and Σ_{12} and Σ_{21}.
   - (5 pts) Plot a *scatter plot* of $\mathbf{X}^{(1)}$. Suppose the variable `x` is a $2 \times N$ variable, with the first row containing samples from the first element of $\mathbf{X}^{(1)}$ and the second row containing samples from the second element of $\mathbf{X}^{(1)}$. The scatter plot can be obtained with

```
plot(x(1,:),x(2,:),'.','MarkerSize',1);
```

That is, it plots the first row against the second row, putting a $\cdot$ at each coordinate point.

Python:

```
plt.scatter(X[0,:], X[1,:], s = 0.5, marker = '.')
```

Notice from the contour plot or the scatter plot that a simple rotational change of coordinates will suffice to to make the elements be uncorrelated (and, since Gaussian, independent).

- (15 pts) Suppose that a measurement of the three variables x_1, x_2, and x_3 is available. (5 pts) Write a function `predictx4` which will estimate the corresponding value of x_4. The function should have the "declaration": Matlab:

  ```
  function [x4hat,sigma_4squared] = predictx4(x1,x2,x3)
  ```

  Python:

  ```
  def predict4(x1,x2,x3)
  ```

  (with possibly some other arguments as well).

  Provide a listing of your code.

  If $x_1 = 3$, $x_2 = 3.6$, $x_3 = 5.2$:
  - (3 pts) Given this data, what is the estimate of x_4?
  - (4 pts) Given this data, what is the estimate of the variance of the estimate?
  - (3 pts) Explicitly state the covariance and mean vectors you use in your computations.
- (10 pts) Suppose now that $x_1 = 5$ and $x_3 = 7$ are measured, and x_2 and x_4 are to be estimated.

  (5 pts) Write a function `predict24` which estimates x_2 and x_4, given x_1 and x_3, and which also estimates the covariance of the estimate. (Provide a listing of your code.)
  - (5 pts) Given the values of x_1 and x_3 above, determine the best estimate of x_2 and x_4.
  - (3 pts) Determine the covariance of x_2 and x_4, given x_1 and x_3.
  - (2 pts) What are the estimated values of $\boldsymbol{\mu}^{(1)}$, $\boldsymbol{\mu}^{(2)}$, Σ_{11}, Σ_{21}, etc.?

Transformations of Random Variables

1. (10 pts) Write a MATLAB or Python function `myrandn` that will generate a single standard Gaussian random variable $\mathcal{N}(0,1)$ using uniform $\mathcal{U}[0,1]$ random variables and the Box-Muller transformation. Validate that it generates a Gaussian by plotting a histogram of output data. Estimate the mean and variance, and compare with the desired mean and variance.
2. (5 pts) Write a MATLAB or Python function that will generate an exponential $\mathcal{E}(1)$ random variable, using transformation of variables. Verify the distribution by generating many data points and plotting the data histogram. Find the sample mean and compare with the theoretical mean.
3. (15 pts) Let $X \sim \mathcal{U}(-\pi, \pi)$, and let $Y = \sin(X)$.

   (a) (5 pts) Determine the pdf of Y.

(b) (5 pts) Generate 10000 points of X data and from this, 10000 points of Y data. Plot the histogram of Y with 100 bins. Scale the histogram so it has area 1, like a pdf.

```
histogram(Y,100,'normalization','pdf);
hold on;                    % hold for future plotting
```

(c) (5 pts) Compare the histogram of Y with the pdf of Y by plotting the pdf of Y on the same plot as the histogram.

Blind Source Separation This topic is also called **independent component analysis**.

This section contains a lot of background, with a few exercises.

- (5 pts) Make a scatter plot of two independent $\mathcal{U}(-1,1)$ random variables with the following code:

```
N = 1000;
s1 = 2*rand(1,N)-1;
s2 = 2*rand(1,N)-1;
plot(s1,s2,'.','MarkerSize',1);
axis([-5 5 -5 5]);
axis('equal')
```

Python:

```
N = 1000
s1 = 2*np.random.rand(N)-1
s2 = 2*np.random.rand(N)-1
plt.scatter(s1,s2,s=0.5, marker='.')
plt.axis((-5,5,-5,5))
```

Independence of the random variables is evident from the shape of the scatter plot – the boundary of s1 is independent of the boundary of s2 (the boundaries align with the coordinate axes).

- (5 pts) Now transform the data and plot it:

```
A = [2.5 2; 1 3];
y = A*[s1;s2];
x1 = y(1,:);
x2 = y(2,:);
plot(x1,x2,'.','MarkerSize',1);
axis([-5 5 -5 5]);
axis('equal')
```

It is clear now that no simple rotation of the axes will make the variables independent.

Python:

```
A = np.array([[2.5, 2], [1,3]])
y = A @ np.vstack((s1,s2))
plt.scatter(y[0,:],y[1,:], s=0.5, marker = '.')
plt.axis((-5, 5, -5, 5))
```

Consider now the following problem: Let $S_1(t), S_2(t), \ldots, S_n(t)$ denote a sequence of independent random processes (signals), assumed for convenience to be zero mean. That is, at each time t, the random variables are independent. (Think of samples of speech in a room.) These are mixed together by a square matrix A, assumed to be invertible, to produce the samples $X_1(t), X_2(t), \ldots, X_n(t)$:

$$\begin{bmatrix} X_1(t) \\ X_2(t) \\ \vdots \\ X_n(t) \end{bmatrix} = A \begin{bmatrix} S_1(t) \\ S_2(t) \\ \vdots \\ S_n(t) \end{bmatrix}.$$

Let $\mathbf{X}(t)$ and $\mathbf{S}(t)$ denote the vectors

$$\begin{bmatrix} X_1(t) \\ X_2(t) \\ \vdots \\ X_n(t) \end{bmatrix} \qquad \begin{bmatrix} S_1(t) \\ S_2(t) \\ \vdots \\ S_n(t) \end{bmatrix},$$

respectively, so that we can write

$$\mathbf{X}(t) = A\mathbf{S}(t).$$

The problem can now be stated: Given the sequence of vectors $\mathbf{X}(t)$, $t = 1, 2, \ldots,$, recover the original data $\mathbf{S}(1), \mathbf{S}(2), \ldots$, *without* knowing the mixing matrix A. That is, we want to find a matrix B so that

$$\mathbf{Y}(t) = B\mathbf{X}(t).$$

where $\mathbf{Y}(t) = \mathbf{S}(t)$. Of course, if we knew A, we could simply set $B = A^{-1}$. But we don't know A. Sounds impossible, no? Actually, as just stated, it is. We allow ourselves the possibility that the elements of $\mathbf{Y}$ are permuted and scaled versions of the elements of $\mathbf{S}$. This still sounds impossible, but it is not.

We know something about the original data: the samples were indpendent. But (as seen in the example above) the mixed data are not in general independent. The approach, then, is to find a matrix B somehow which forces the elements of $\mathbf{Y}(t) = B\mathbf{X}(t)$ to be statistically independent.

Question: Could we find a rotation matrix to do the job? That is, could we simply whiten (that is, decorrelate) the data? Answer: No. Simply rotating the coordinate system does not suffice to make the components independent.

Our approach is to establish a *parametric model* for the data, then find the parameters which best fit the model. This best description of the data, as it turns out, also serves to make the components of $\mathbf{Y}(t)$ independent.

For random vectors $\mathbf{X}$ and $\mathbf{S}$, write

$$\mathbf{X} = A\mathbf{S}.$$

Then $\mathbf{S} = A^{-1}\mathbf{X}$. Let $B = A^{-1}$. Then

$$\frac{\partial \mathbf{S}}{\partial \mathbf{X}^T} = B,$$

and the Jacobian is $|\det(B)|$. Here,

$$\frac{\partial \mathbf{S}}{\partial \mathbf{X}^T}$$

represents the matrix of partial derivatives,

$$\frac{\partial \mathbf{S}}{\partial \mathbf{X}^T} = \begin{bmatrix} \frac{\partial s_1}{\partial x_1} & \frac{\partial s_1}{\partial x_2} & \cdots & \frac{\partial s_1}{\partial x_n} \\ \frac{\partial s_2}{\partial x_1} & \frac{\partial s_2}{\partial x_2} & \cdots & \frac{\partial s_2}{\partial x_n} \\ \vdots & & & \\ \frac{\partial s_n}{\partial x_1} & \frac{\partial s_n}{\partial x_2} & \cdots & \frac{\partial s_n}{\partial x_n} \end{bmatrix}$$

Then we know that the density of $\mathbf{X}$ can be written in terms of the density of $\mathbf{S}$ as

$$f_{\mathbf{X}}(\mathbf{x}) = f_{\mathbf{S}}(\mathbf{s})|\det(B)||_{\mathbf{s}=B\mathbf{x}}\,.$$

Let us write B as a stack of row vectors,

$$B = \begin{bmatrix} \mathbf{b}_1^T \\ \mathbf{b}_2^T \\ \vdots \\ \mathbf{b}_n^T \end{bmatrix}.$$

Then $s_i = \mathbf{b}_i^T\mathbf{x}$. Under the assumption that the elements of $\mathbf{S}$ are independent (but not necessarily identically distributed) we can write

$$f_{\mathbf{X}}(\mathbf{x}) = |\det(B)| \prod_{i=1}^{n} f_{S_i}(s_i)\Big|_{s_i=\mathbf{b}_i^T\mathbf{x}}.$$

Now let us suppose that we have N indepdendent observations of the $\mathbf{X}$ data, for $t = 1, 2, \ldots, N$. That is,

$$\mathbf{X}(1) = \mathbf{x}_1, \quad \mathbf{X}(2) = \mathbf{x}_2, \cdots, \mathbf{X}(N) = \mathbf{x}_N$$

The likelihood of the observation $\mathbf{x}_t$ can be computed as

$$f_{\mathbf{X}}(\mathbf{x}_t) = |\det(B)| \prod_{i=1}^{n} f_{S_i}(\mathbf{b}_i^T\mathbf{x}_t), \qquad i = 1, 2, \ldots, N.$$

The *likelihood function* for the entire sequence is the product of each of these likelihoods. We denote this as $L(B|\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_N)$. This notation emphasizes that to compute this likelihood, we need to know the matrix B. But it also depends upon the observed data $\mathbf{x}_i$. We thus have

$$L(B|\mathbf{x}_1, \ldots, \mathbf{x}_N) = \prod_{t=1}^{N} \prod_{i=1}^{n} f_{S_i}(\mathbf{b}_i^T\mathbf{x}_t)|\det(B)|.$$

As an abbreviated notation, we will use $L(B)$ in place of $L(B|\mathbf{x}_1, \ldots, \mathbf{x}_T)$ (since B is the variable that interests us here).

We want to find the matrix B which best fits the sequence of observations, which is the one which gives the highest probability for the observations. That is, we want to maximize the likelihood,

$$B = \arg\max_B L(B).$$

This is the *maximum likelihood parameter estimation approach*. We now turn our attention to the maximization.

It will turn out to be more convenient to compute the logarithm of the likelihood function. This makes no difference to the optimization problem: if $L(B)$ is maximized, then $\log L(B)$ is maximized. The logarithm conventiently turns products into sums:

$$\log L(B) = \sum_{t=1}^{N} \sum_{i=1}^{n} \log f_{S_i}(\mathbf{b}_i^T \mathbf{x}_t) + N \log |\det(B)|.$$

Normalizing by $1/N$, we will represent the sum over t as an average. We will also use a shorthand notation, as if the average were, in fact, equal to the expectation.

$$\begin{aligned}\frac{1}{N} \log L(B) &= \frac{1}{N} \left[\sum_{t=1}^{N} \sum_{i=1}^{n} \log f_{S_i}(\mathbf{b}_i^T \mathbf{x}_t) + N \log |\det(B)| \right] \\ &= E[\sum_{i=1}^{n} \log f_{S_i}(\mathbf{b}_i^T \mathbf{x})] + \log |\det(B)|. \qquad \text{(write the average as if it were an expectation)}\end{aligned}$$

So, how to find the optimizing B? The approach that we take, which is very common in the signal processing literature, is to use *gradient ascent*. The function we want to maximize is $\frac{1}{N} \log L(B)$. We compute the gradient of this function with respect to the parameter matrix B, then move in the direction of this gradient. Letting $B^{[k]}$ denote the value of the B matrix we have at the kth iteration, we take

$$B^{[k+1]} = B^{[k]} + \mu \frac{\partial}{\partial B} \left(\frac{1}{N} \log L(B) \right).$$

For an appropriately small *stepsize* μ, every iteration of this update will increase the log likelihood function. In practice, we use each observed data point $\mathbf{x}_1, \mathbf{x}_2, \ldots,$ in succession, working through all the data. If necessary (for short data records) we can sweep multiple times through the data.

What we mean by the gradient with respect to a matrix B is the matrix of partial derivatives, with the same shape and orientation as the matrix B:

$$\frac{\partial}{\partial B} = \begin{bmatrix} \frac{\partial}{\partial b_{11}} & \frac{\partial}{\partial b_{12}} & \cdots & \frac{\partial}{\partial b_{1n}} \\ \frac{\partial}{\partial b_{21}} & \frac{\partial}{\partial b_{22}} & \cdots & \frac{\partial}{\partial b_{2n}} \\ \vdots & & & \\ \frac{\partial}{\partial b_{n1}} & \frac{\partial}{\partial b_{n2}} & \cdots & \frac{\partial}{\partial b_{nn}} \end{bmatrix}$$

The remaining difficulty is to compute the gradient

$$\frac{\partial}{\partial B}\frac{1}{N}\log L(B) = \frac{\partial}{\partial B}E[\sum_{i=1}^{n}\log f_{S_i}(\mathbf{b}_i^T\mathbf{x})] + \log|\det(B)|.$$

It is a standard result that

$$\frac{\partial}{\partial B}\log|\det(B)| = (B^{-1})^T = B^{-T}.$$

Also, it can be shown that (that is, this is what you get to show)

$$\frac{\partial}{\partial B}E[\sum_{i=1}^{n}\log f_{S_i}(\mathbf{b}_i^T\mathbf{x})] = E[\mathbf{g}(B\mathbf{x})\mathbf{x}^T] \tag{1}$$

where $\mathbf{g}$ is the vector function

$$\mathbf{g}(\mathbf{y}) = \begin{bmatrix} g_1(y_1) \\ g_2(y_2) \\ \vdots \\ g_n(y_n) \end{bmatrix}$$

whose components are

$$g_i(y_i) = (\log f_{S_i}(y_i))' = \frac{f'_{S_i}(y_i)}{f_{S_i}(y_i)}$$

and where f'_{S_i} is the derivative of the function f_{S_i}.

- (15 pts) Show that (1) is true.

Putting the pieces together, we find

$$\frac{\partial}{\partial B}\frac{1}{N}\log(B) = B^{-T} + E[\mathbf{g}(B\mathbf{x})\mathbf{x}^T]$$

This is a little complicated, since at every stage the inverse B^{-1} must be computed. While it is not clear here (and would take some doing to justify this) the algorithm has lower complexity, converges faster, and is more stable if we multiply this gradient by the matrix B^TB:

$$\frac{\partial}{\partial B}\frac{1}{N}\log(B)B^TB = (B^{-T} + E[\mathbf{g}(B\mathbf{x})\mathbf{x}^T])B^TB = (I + E[\mathbf{g}(B\mathbf{x})\mathbf{y}^T])B.$$

Here, I is the $n \times n$ identity matrix. We will use this in our update rule.

One last issue need to be addressed. To truly compute the expectation $E[\mathbf{g}(B\mathbf{x})\mathbf{y}^T]$ we would need some kind of probability to average over. But recall that this was simply a shortand for the summation over t. We can therefore drop the expectation.

We thus obtain with matrix update rule

$$\boxed{B^{[k+1]} = B^{[k]} + \mu(I + \mathbf{g}(B\mathbf{x})\mathbf{y})B^{[k]}.}$$

The algorithm can be summarized as follows:

Maximum Likelihood Independent Component Analysis

Input: Data sequence $\mathbf{x}_1, \ldots, \mathbf{x}_N$, Stepsize μ. Initial matrix $B^{[1]}$.
for $t = 1 : N$

$\mathbf{y}_t = B^{[t]}\mathbf{x}_t$. (compute the output for the current input)
$B^{[t+1]} = B^{[t]} + \mu(I + g(\mathbf{y}_t)\mathbf{y}_t^T)B^{[t]}$ (update the separating matrix)
end for

So, it took a lot of work to get here, but turns out to be pretty easy.

There is only one remaining problem. The elements of the $\mathbf{g}$ function are, again,

$$g_i = \frac{f'_{S_i}}{f_{S_i}}.$$

That is, it is the ratio of the derivative of the pdf of S_i over the pdf of S_i. But what if we don't have an expression for the pdf of S_i? All we assumed initially is that the elements of $\mathbf{S}$ are independent; we never assumed that we knew the pdf of $\mathbf{S}$!

At this point, an interesting fact comes in to play: We don't need to know the densities after all! If we only a little bit of information about the density, we can just throw in an approximate function that just the job just fine.

A random variable is said to be *supergaussian*, loosely speaking, if its pdf f decays more slowly than the Gaussian density. Since a Gaussian decays as e^{-x^2} — that is, as the exponent of the square of its argument, a supergaussian is one which decays more slowly than this. A Laplacian density, which decays as $e^{-|x|}$, or an exponential, which goes as e^{-x} (positive x) decays more slowly than a Gaussian; these are both supergaussian densities.

A random variable is said to be *subgaussian*, loosely speaking, if its pdf f decays faster than a Gaussian. The common example is a uniform pdf: outside of its range it decays immediately to zero.

Now, why do we care about supergaussian and subgaussian? The answer is this: we don't have to use the exact function $g = f'_{S_i}/f_{S_i}$, we just have to get close enough. "Close enough" here means that we can use one function g for *any* supergaussian, and another function g for *any* subgaussian. There is, in each case, considerable latitude in how the functions are selected. However, there are some commonly used functions. For a **supergaussian density**, the function

$$g(y) = -2\tanh(y)$$

is frequently used. For a **subgaussian density** the functions

$$g(y) = \tanh(y) - y$$

or

$$g(y) = -y^3$$

are frequently used.

- (10 pts) Code the maximum likelihood independent component algorithm. You may want to have a function declaration such as

  ```
  function [y,B] = bss(x,B0,mu)
  ```

  Python:

  ```
  def bss(x,B0,mu)
  ```

  This function iterates through each $\mathbf{x}_t$ (that is, each column of `x`) separately (so it has its own for loop, iterating over the length of the $\mathbf{x}$ vector).

  Allow the possibility of iterating multiple times over the data, such as

  ```
  for i=1:Numiterations
    [y,W] = bss(x,W0,mu);
    W0 = W;
  end
  ```

  Python:

  ```
  for i in range(Numiterations):
      (y,W) = bss(x,W0,mu)
      W0 = W
  ```

  Apply your algorithm to the random $\mathbf{x}$ data you created above, and plot the scatter plot of the allegedly separated data $\mathbf{y}$ after each pass through the data. Does it in fact lead to independent data? Try various values of μ, such as $\mu = 0.0001$, $\mu = 0.001$, etc.
- (20 pts) The data files `x1.mat`, `x2.mat`, and `x3.mat` contain the data $x_1(t)$, $x_2(t)$, and $x_3(t)$, which are mixtures of 3 audio signals. Play each of these files using the `sound` command; you can hear the mixture.

  ```
  % read the data in
  load x1
  load x2
  load x3
  x = [x1;x2;x3];   % stack up the data
  fprintf(1,'Play the original mixed data\n');
  for i=1:3
    fprintf('mixed signal %d\n',i);
    sound(x(i,:))
  end
  ```

  Python: There are a lot of ways to play audio from Python, usually from audio files. The **sounddevice** package seemed to work for me. You'll need to install it before using it:

  ```
  pip install sounddevice
  ```

  Load the files. The data is in floating point format, so you'll need to convert it to integer.

```
import numpy as np
from scipy.io import loadmat
import sounddevice as sd

data = loadmat('x1.mat')
x1 = data['x1'][0]    # these are floating point in the range [-1,1]
# x1int = (x1*32768).astype(np.int16)   # if your sound device needs
data = loadmat('x2.mat')
x2 = data['x2'][0]
# x2int = (x2*32768).astype(np.int16)
data = loadmat('x3.mat')
x3 = data['x3'][0]
# x3int = (x3*32768).astype(np.int16)
x = np.vstack((x1,x2,x3))  # stack up the data

for i in range(3):
    print(f'mixed signal {i}')
    sd.play(x[i,:],8000)
    sd.wait()
```

Now stack the data as appropriate to make vectors, and run through your blind source separation algorithm. Play the result with the `sound` command. You should hear how the separation occurs. Try different values of μ and running through the data different numbers of times. Try both a subgaussian function g and a supergaussian function g. Which works better to separate the speech signal? Provide a description of how the algorithm converges (how long does it take to converge? how well separated is it? what values of μ work best? etc.)

```
fprintf('separating the data...');
W0 = eye(3);
mu=0.001;   % or other values ...
Niter = 4;  % other other values  ...
for i=1:Niter
  [y,W] = bss(x,W0,mu);
  W0 = W;
end

fprintf('Playing the separated data\n');
for i=1:3
  fprintf('separated signal %d\n',i);
  % normalize the data to avoid overdriving
  out = y(i,:)/(1.1*max(abs(y(i,:))));
  sound(out);
end
```

Python:

```
W0 = np.identity(3)
mu = 0.001  # other other values
Niter = 4   # number of iterations through the data
for i in range(Niter):
   (y,W) = bss(x,W0,mu)
   W0 = W

print('Playing the separated data')
for i in range(3):
    print(f'separated signal {i}')
    # normalize the data
    out = y[i,:]/(1.1*np.max(abs(y[i,:])))
    sd.play(out)
    sd.wait()
```

In your writeup, describe how well the separation works, what you learned, how the algorithm changes when you change μ, and anything else of interest.

Turn in your plots, hand calculations, programs, observations, and conclusions.